Library Management System
DBMS

Student: Amalia-Andreea Moisi
Study Program: Computer Science

Abstract

This document outlines the requirements and database design for a comprehensive Library Management System (LMS). The primary objective of this project is to develop a centralized, efficient platform that streamlines both patron interactions and administrative cataloging. The proposed system facilitates user account management, allows patrons to browse an organized catalog of literature, and handles the complete lifecycle of book borrowing. By establishing a strictly relational database architecture, the system accurately tracks detailed metadata for authors, books, and distinct physical or digital copies, while seamlessly managing transactional data such as active loans, borrowing histories, and item reservations.

Table of Contents

Abstract _____ 1

Chapter 1:

Introduction _____ 3

Requirements Analysis _____ 3

Chapter 2:

Database Design, ER Diagram _____ 5

Functional Diagram _____ 6

Normalization _____ 7

Chapter 3:

Database Implementation _____ 8

Chapter 4:

SQL Query Analysis _____ 10

Chapter 5:

Database Performance Discussion _____ 11

Chapter 6:

Conclusions _____ 13

References _____ 14

Chapter 1

Introduction

Modern libraries require robust data management solutions to maintain accurate inventories and provide a seamless experience for their users. The proposed Library Management System addresses this core business need by replacing fragmented tracking methods with a structured, relational database environment.

The system is designed with two primary user groups in mind: the library administrators (or librarians) who maintain the collection, and the clients who utilize it. To achieve this, the system focuses on several key functional objectives:

- **Catalog Management:** Providing a highly structured catalog that organizes literature by author, book, and the specific physical or digital copies available, utilizing standard metadata such as publication year and ISBN.
- **User Account Administration:** Allowing members to manage their personal accounts while enabling the system to securely track their current status and historical data.
- **Circulation and Transactions:** Facilitating the core operations of a library by processing book loans, tracking due dates, and maintaining an accurate borrowing history.
- **Reservation Handling:** Enhancing the user experience by allowing members to place reservations (holds) on items that are currently checked out or unavailable.

Ultimately, this system aims to ensure data integrity, reduce redundancy, and provide a reliable infrastructure for day-to-day library operations.

Requirements Analysis

Entities:

- **Users:** The registered clients who interact with the library to manage their accounts and borrow materials.
- **Authors:** The creative entities responsible for writing the literary content.
- **Books:** The distinct intellectual works organized and cataloged by the library.
- **Copies:** The individual physical or digital units available for checkout, reading, or reservation.
- **Reservations:** Custom requests made by a member to hold a specific book when it becomes available.
- **Loans:** The transactional records capturing a member's borrowing history, due dates, and return status.

Attributes:

- **Users:** *UserID, FirstName, LastName, Email, JoinDate, Address
- **Authors:** *AuthorID, Name, Biography, Nationality
- **Books:** *BookID, Title, PublicationYear, Genre, ISBN, AuthorID
- **Copies:** *CopyID, ConditionStatus, Format, Status, BookID
- **Reservations:** *ReservationID, ReservationDate, Status, UserID, BookID
- **Loans:** *LoanID, LoanDate, DueDate, Status, UserID

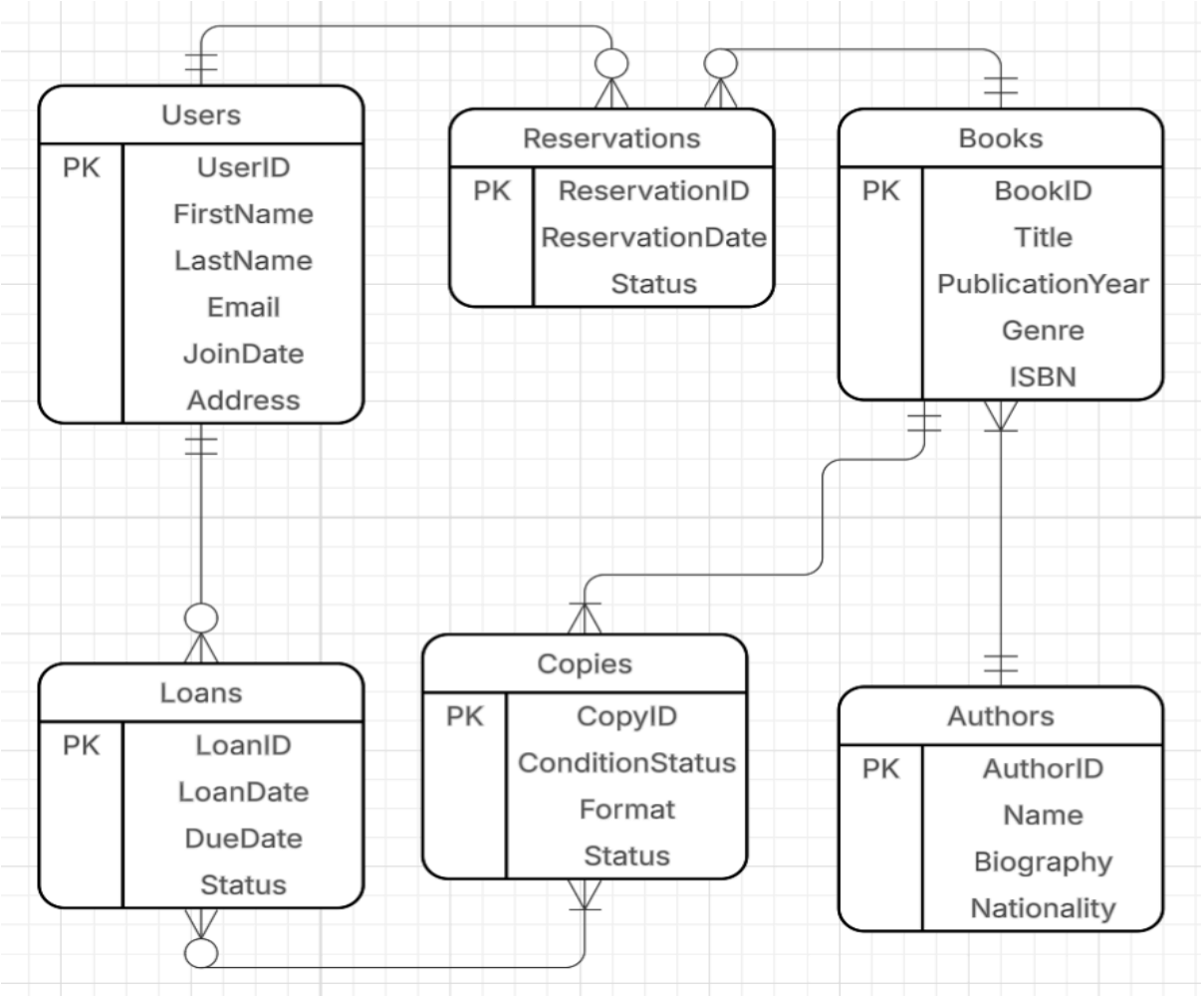
Relationships:

- A user can make multiple loans. A loan must belong to only one user.
- A user may create multiple reservations. A reservation must belong to only one user.
- An author must write books. Each book must belong to one and only one author.
- A book must contain at least one copy. Each copy must belong to only one book.
- A reservation must target one book. A book can have multiple reservations.
- A loan must contain copies. A copy can appear in multiple loans over time.

Chapter 2

Initial ERD(Conceptual Model)

In the conceptual phase, entities are linked logically. The many-to-many relationships exist directly between entities without junction tables.



Relational Model

To map the conceptual ERD into a relational database, we resolved the many-to-many relationships by introducing bridges. Below is the mapped relational model.

- Users(UserID, FirstName, LastName, Email, JoinDate, Address)
- Reservations(ReservationID, ReservationDate, Status, BookID(fk), UserID(fk))
- Books(BookID, Title, PublicationYear, Genre, ISBN, AuthorID(fk))
- Loans(LoanID, LoanDate, DueDate, Status, UserID(fk))
- Copies(CopyID, ConditionStatus, Format, Status, BookID(fk))
- Authors(AuthorID, Name, Biography, Nationality)
- Loan_Copy(LoanID(fk), CopyID(fk))

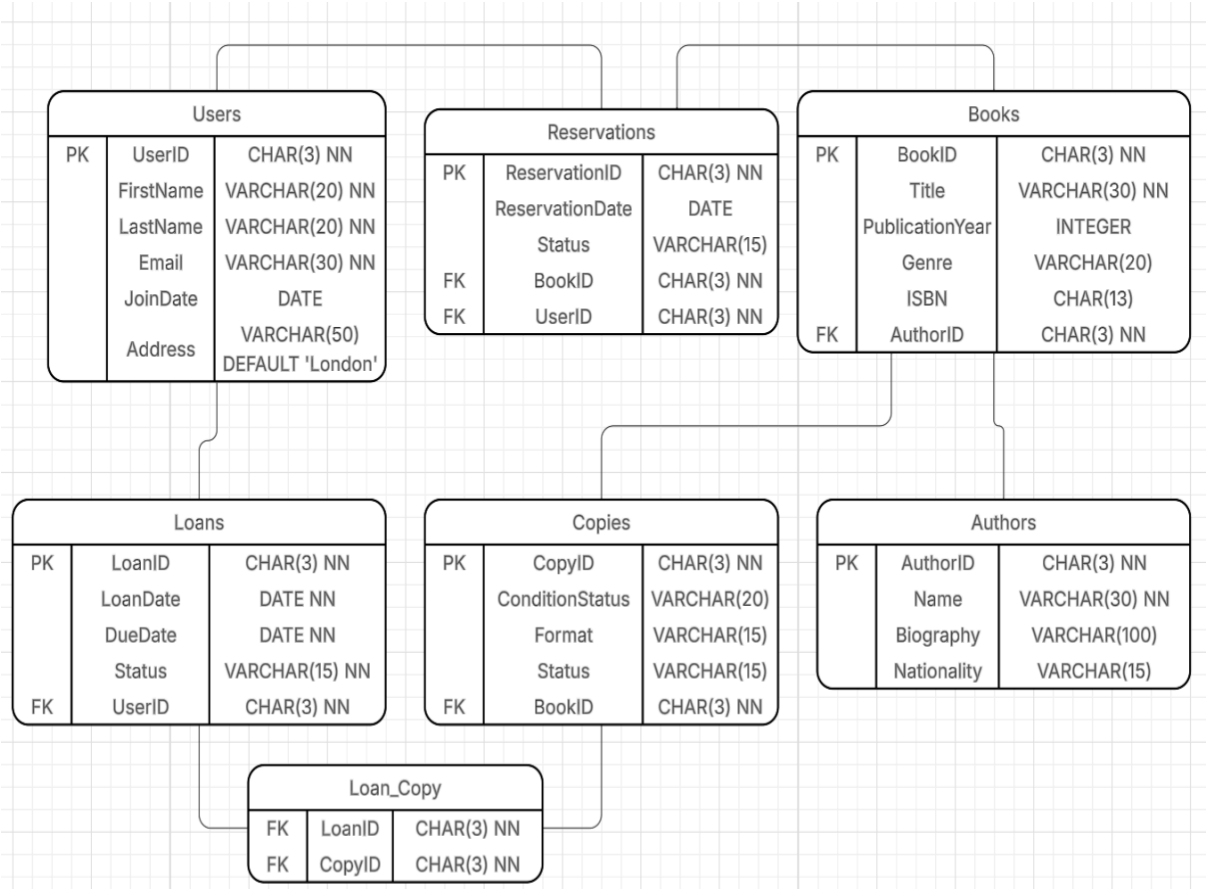
Complete Functional Dependencies

Based on our Library Management System model, the functional dependencies are for:

- **Users Entity:** UserID→FirstName, LastName, Email, JoinDate, Address
- **Authors Entity:** AuthorID→Name, Biography, Nationality
- **Books Entity:** BookID→Title, PublicationYear, Genre, ISBN, AuthorID
- **Copies Entity:** CopyID→ConditionStatus, Format, Status, BookID
- **Reservations Entity:** ReservationID→UserID, BookID, ReservationDate, Status
- **Loans Entity:** LoanID→UserID, CopyID, LoanDate, DueDate, Status

Note: In the Book, Copy, Reservation, and Loan entities, the foreign keys (like AuthorID in the Books entity) are functionally dependent on the primary key of that specific table, which is what links the relational model together.

Final Functional Diagram



Normalization

How Redundancy Was Reduced

By structuring the database around these strict functional dependencies (ensuring it meets First, Second, and Third Normal Forms), we significantly reduce data redundancy and protect the database from data anomalies. Here is exactly how that was achieved:

1. Elimination of Data Duplication (Update Anomalies) If we did not isolate our entities based on functional dependencies, we might have a single, massive "Checkout" table containing User details, Book details, and Loan details all in one row.

- **The Problem:** Every time a user borrowed a new book, their details would be written into the database again. If that user changed their email address, the database engineer would have to find and update every single loan record associated with that member.
- **The Solution:** By enforcing the dependency UerID→FirstName, LastName, Email, Address we only store the user's details once in the User table. If an email changes, we update exactly one cell in one row. This resolves partial dependencies if the massive table had a composite key, moving the database into Second Normal Form (2NF). 1NF already being achieved as we do not have rows with multiple values.

2. Independent Data Creation (Insertion Anomalies) Redundancy often forces us to have incomplete data just to create a record.

- **The Problem:** If Author data was stored inside the Book table, we could not add a new author to our system until they had officially published a book that our library purchased.
- **The Solution:** By separating the dependencies (AuthorID→Name, Bio, Nationality distinct from BookID→Title, PublicationYear, Genre, ISBN), we can insert an Author into the database independently of whether we currently hold any of their Books. This eliminates transitive dependencies, where Author details mistakenly depend on the BookID rather than directly on the AuthorID, ensuring the database meets Third Normal Form (3NF).

3. Safe Data Removal (Deletion Anomalies) When data is redundant and grouped incorrectly, deleting one piece of information can accidentally wipe out other vital information.

- **The Problem:** If book metadata (Title, PublicationYear) was stored directly on the Copy entity, discarding the library's last worn-out physical copy of "1984" would permanently delete all records that the book "1984" ever existed in the library's system.
- **The Solution:** By separating CopyID→ConditionState from BookID→Title, we can safely delete a damaged Copy record without losing the underlying Book catalog information. The book remains in the catalog, signaling to the librarians that they need to order a replacement copy. This is another example of achieving Third Normal Form (3NF) by stripping out transitive dependencies.

Chapter 3

Database Implementation

1. Database & Table Creation

- **CREATE DATABASE**: Sets up the overall database, the container that will hold all the tables, views, and database objects.
- **CREATE TABLE**: Defines the individual entities (eg. Users, Books, or Loans) and specifies the columns each table will contain alongside their data types and constraints.

2. Constraints (Data Rules)

Constraints enforce rules on the data to ensure integrity and accuracy so unfit data doesn't get into the system:

- **Primary Key**: Uniquely identifies each record in a table (eg. UserID). It cannot contain NULL values.
- **Foreign Key**: Links two tables together by referencing the PRIMARY KEY of another table (eg. linking BookID in Copies back to Books). This maintains relational integrity.
- **Not Null**: Prevents a column from accepting empty (NULL) values (eg, forcing every user to have a Name and an Email).
- **Default Values**: Assigns a fallback value automatically if no value is explicitly provided during an INSERT (like defaulting Address to 'London').

3. Data Types

- **Text/Strings**:
    - CHAR(n) for fixed lengths like IDs and ISBN
    - VARCHAR(n) for variable lengths like Name, FirstName
- **Numbers**: INT for integers like PublicationYear
- **Dates**: DATE for attributes like LoanDate, DueDate

```sql
CREATE DATABASE Library_Management_System;
USE Library_Management_System;
CREATE TABLE Users(
    UserID CHAR(3) PRIMARY KEY,
    FirstName VARCHAR(20) NOT NULL,
    LastName VARCHAR(20) NOT NULL,
    Email VARCHAR(30) NOT NULL,
    JoinDate DATE,
    Address VARCHAR(50) DEFAULT 'London'
);
CREATE TABLE Authors(
    AuthorID CHAR(3) PRIMARY KEY,
    Name VARCHAR(30) NOT NULL,
    Biography VARCHAR(100),
    Nationality Varchar(15)
);
CREATE TABLE Books(
    BookID CHAR(3) PRIMARY KEY,
    Title VARCHAR(30) NOT NULL,
    PublicationYear INT,
    Genre VARCHAR(20),
    ISBN CHAR(13),
    AuthorID CHAR(3),
    FOREIGN KEY(AuthorID) REFERENCES Authors(AuthorID)
);
```

```sql
INSERT INTO Users VALUES
    ('001', 'Andrei', 'Clerence', 'AndrCl@gmail.com', '2024-07-23', '12095 Didcot'),
    ('002', 'Anabelle', 'Mihovnici', 'AnabelleMihovnici@gmail.com', '2024-05-12', '34123 Clinton'),
    ('003', 'Alice', 'Sanda', 'alice12san@gmail.com', '2022-12-24', DEFAULT),
    ('004', 'Alec', 'Diaz', 'alecccdiazzz@gmail.com', '2022-08-13', DEFAULT),
    ('005', 'Alex', 'Garsia', 'alexandergarsia@gmail.com', '2022-03-09', DEFAULT),
    ('006', 'Elena', 'Popescu', 'elena.popescu@gmail.com', '2023-01-15', DEFAULT),
    ('007', 'Matei', 'Ionescu', 'matei.ionescu@yahoo.com', '2023-03-20', '45 Oxford St'),
    ('008', 'Sofia', 'Radu', 'sofia.radu@outlook.com', '2023-05-11', DEFAULT),
    ('009', 'David', 'Dumitru', 'david.dumitru@gmail.com', '2023-08-02', '12 Baker St'),
    ('010', 'Maria', 'Stan', 'maria.stan@gmail.com', '2023-11-29', DEFAULT),
    ('011', 'Luka', 'Novak', 'luka.novak@gmail.com', '2024-01-10', '88 Piccadilly'),
    ('012', 'Chloe', 'Dubois', 'chloe.dubois@yahoo.fr', '2024-02-14', DEFAULT),
    ('013', 'Oliver', 'Smith', 'oliver.smith@gmail.com', '2024-04-18', '10 Downing St'),
    ('014', 'Emma', 'Taylor', 'emma.taylor@outlook.com', '2024-06-30', DEFAULT),
    ('015', 'Noah', 'Brown', 'noah.brown@gmail.com', '2024-09-05', DEFAULT),
    ('016', 'Liam', 'Wilson', 'liam.wilson@yahoo.com', '2024-10-12', '15 Regent St'),
    ('017', 'Mia', 'Garcia', 'mia.garcia@gmail.com', '2024-12-01', DEFAULT),
    ('018', 'Lucas', 'Martin', 'lucas.martin@outlook.com', '2025-01-19', DEFAULT),
    ('019', 'Aria', 'Johnson', 'aria.johnson@gmail.com', '2025-03-04', '22 Abbey Road'),
    ('020', 'Ethan', 'Davies', 'ethan.davies@gmail.com', '2025-04-22', DEFAULT);

INSERT INTO Authors VALUES
    ('A01', 'Sally Rooney', 'Irish author known for modern relationships novels', 'Irish'),
```

```sql
158
159 •   SELECT FirstName,
160             LastName,
161             Title AS Book_Title,
162             L.Status AS Loan_Status
163     FROM Users AS U
164         LEFT JOIN Loans AS L
165             ON L.UserID=U.UserID
166                 JOIN Loan_Copy AS LC
167                     ON LC.LoanID=L.LoanID
168                         JOIN Copies AS C
169                             ON C.CopyID=LC.CopyID
170                                 JOIN Books AS B
171                                     ON B.BookID=C.BookID;
```

100% 8:157

Result Grid | Filter Rows: Search | Export:

FirstName	LastName	Book_Title	Loan_Status
Andrei	Clerence	Normal People	Returned
Alice	Sanda	Conversations with Friends	Returned
Alex	Garsia	Beautiful World Where Are You	Returned
Elena	Popescu	Good Material	Returned
Matei	Ionescu	1984	Overdue
Sofia	Radu	Animal Farm	Returned
David	Dumitru	Normal People	Active
Luka	Novak	Everything I Know About Love	Active
Oliver	Smith	Pride and Prejudice	Active
Emma	Taylor	The Shining	Active

Chapter 4

SQL Query Analysis

1. Marketing & User Analytics

The system has queries that focus on understanding user behavior and segmenting the audience, like: latest gmail users (query 1), loyalty ranking (query 5), reading preferences (query 6) and exclusive readers (query 7).

2. Inventory & Catalog Management

These queries help librarians understand the state and composition of their collection: Prolific Authors (query 2), Targeted Genre Report (query 3), Condition Categorizing Books (query 4), Age Spread per Genre (query 10).

3. Operations & Automations

These queries move beyond basic data retrieval and focus on database architecture and administrative efficiency: Temporary Table for Top Readers (query 8), Overdue Loans View (query 9), Reservation Limit Trigger (query 11), New Book Stored Procedure (query 12).

For once, the stored procedure (query 12) automates a two-step book intake process by taking the book details as input parameters and first inserting a new record into the Books table. It then retrieves the auto-generated primary key using LAST_INSERT_ID() and uses it to automatically create a corresponding record in the Copies table.

```
-- 12.
/*When a new book arrives at the library, staff currently have to manually insert
the book record, find the new BookID, and then manually insert a copy record.
We need a Stored Procedure to automate this workflow into a single call.*/
DELIMITER //
CREATE PROCEDURE RegisterNewBookAndCopy(
    IN New_Title VARCHAR(30),
    IN New_PubYear INT,
    IN New_Genre VARCHAR(20),
    IN New_ISBN CHAR(13),
    IN New_AuthorID CHAR(3),
    IN New_Format VARCHAR(20)
)
BEGIN
    DECLARE New_BookID INT;
    INSERT INTO Books(Title, PublicationYear, Genre, ISBN, AuthorID)
        VALUES(New_Title, New_PubYear, New_Genre, New_ISBN, New_AuthorID);
    SET New_BookID=LAST_INSERT_ID();
    INSERT INTO Copies(ConditionStatus, Format, Status,BookID)
        VALUES('New',New_Format, 'Available', New_BookID);
END;
//DELIMITER ;
```

Chapter 5

Database Performance Discussion

1. Factors Affecting Database Performance

Several key factors determine how efficiently a database executes queries and manages data:

- **Indexing:** The presence (or lack) of indexes on frequently searched columns or foreign keys drastically affects lookup times.
- **Query Design:** How queries are written (eg, using JOIN vs correlated subqueries, or relying on wildcards like LIKE '%@gmail%') impacts execution plans.
- **Database Schema & Normalization:** Well-structured tables reduce redundancy, but over-normalization can lead to excessive, expensive JOIN operations.
- **Hardware and Configuration:** Available RAM for caching, disk I/O speeds (SSD vs. HDD), and CPU resources limit maximum throughput.

2. Performance with Millions of Records

If the library system scaled to millions of users, books, and transactions, the current script would encounter significant bottlenecks:

- **Full Table Scans:** Query #1 uses WHERE Email LIKE '%@gmail%'. Because of the wildcard, the database cannot use an index on the Email column. With millions of users, this would force a slow, full table scan every time it runs.
- **Correlated Subqueries:** Query #10 (which pulls the oldest and newest copy per genre) uses subqueries in the SELECT clause (SELECT MIN(PublicationYear) ... WHERE B_old.Genre=B.Genre). For millions of books, this executes the subquery repeatedly for every row, which could cause the database to time out. Refactoring this to a standard GROUP BY Genre would be highly recommended.
- **Missing Indexes:** The schema relies solely on Primary Keys. To handle scale, we would need to manually create indexes on heavily filtered columns like Status in Reservations, DueDate in Loans, and Genre in Books.

3. The Importance of ACID Compliance

ACID properties ensure that database transactions are processed reliably, which is crucial for a system managing inventory and user accounts. Here is how they apply in this project:

- **Atomicity** ensures that a transaction is treated as a single, indivisible unit that either succeeds completely or fails completely.
    - *Project Example:* Query #12 inserts a record into Books, then into Copies. If the system crashes right after the Books insert, a book will have no physical copies. However, in the insert statements, if one of the rows would contain an error (eg. duplicate values for a Primary key) the entire statement would be aborted.

- **Consistency** ensures that any transaction brings the database from one valid state to another, maintaining all defined rules and constraints.
    - *Project Example:* PreventExcessiveReservations trigger is a perfect example of enforcing business consistency. It ensures the database never enters an invalid state where a user has more than 3 active reservations.
- **Isolation** ensures that concurrent execution of transactions leaves the database in the same state as if they were executed sequentially.
    - *Project Example:* If two users try to place a reservation for the exact same physical CopyID at the exact same millisecond, isolation ensures one transaction completes first, allowing the system to update the copy's status to 'Reserved' and preventing a double-booking.
- **Durability** guarantees that once a transaction has been committed, it will remain committed even in the case of a system failure.
    - *Project Example:* When a librarian marks a loan as 'Returned' in the Loans table and the transaction completes, that data is written to non-volatile storage. If the server loses power a second later, the book will still be marked as returned when the system reboots.

4. The CAP Theorem

The CAP theorem states that distributed data systems can only simultaneously provide two of the following three guarantees:

- **Consistency:** All nodes receive the same copies of a replicated data. In this library system, if a book is checked out, every subsequent query across any database node must instantly reflect that it is checked out.
- **Availability:** Every request received must receive a response, even if the response contains outdated data. This means the library system is always up and accepts operations, even if it temporarily shows a book as available when it was just borrowed elsewhere.
- **Partition Tolerance:** The system can continue operating even if the network connecting the nodes has a fault that results in two or more partitions, where the nodes in each partition can only communicate among each other.

Chapter 6

Conclusions

In conclusion, this project successfully designed and implemented a robust, relational database for a comprehensive Library Management System. By adhering to strict normalization principles up to the Third Normal Form (3NF), the database architecture effectively minimizes data redundancy and protects against update, insertion, and deletion anomalies.

The transition from a conceptual ER diagram to a fully functional SQL implementation demonstrates the system's capability to securely manage user accounts, organize catalog inventory, and process transactional data such as loans and reservations. Furthermore, the integration of advanced SQL queries and the evaluation of database performance ensures that the system is not only structurally sound but also scalable and reliable. Ultimately, this database provides an efficient, centralized foundation that successfully meets the operational needs of library administrators and the everyday needs of its clients.

References

1. Course Materials by Prof. Waje Zerihun
2. Software Tools:
    a. Lucid Chart used for
        i. ER Diagram
        ii. Functional Diagram
    b. MySQLWorkbench used for
        i. database implementation
        ii. query execution
        iii. schema testing
3. External References:
    a. W3Schools SQL Tutorial: https://www.w3schools.com/sql/
    b. The CAP Theorem: https://www.geeksforgeeks.org/dbms/the-cap-theorem-in-dbms/